# Combinator Graph Reduction

A Tutorial Introduction

October 27, 2026

**Abstract**

In 1979 David Turner showed that a functional program can be run without environments, closures or variables of any kind. The program is first translated into a handful of *combinators*, fixed functions such as S, K and I, by a process called bracket abstraction. The result is a tree of applications, which a very simple machine then rewrites, one combinator at a time, until a value remains. Because the tree is rewritten in place, and subtrees are shared rather than copied, it becomes a *graph*, and the machine evaluates lazily: an argument is evaluated only when it is needed, and then only once. This tutorial develops the technique from the beginning. It shows how bracket abstraction works and why Turner's extra combinators make it practical, how the reduction machine finds and rewrites redexes, and how recursion becomes a cycle in the graph. It traces small programs step by step, shows what laziness buys (conditionals without thunks, infinite lists) and what it costs, and then looks at a complete implementation on the VM16 virtual machine with a copying garbage collector, measured against the SECD machine and the Categorical Abstract Machine. The prerequisites are the untyped $\lambda$-calculus and, ideally, the companion tutorials on the SECD machine and the CAM.

## Contents

**1 Introduction** **2**

**2 Combinators** **3**

**3 Bracket abstraction** **3**
3.1 The basic algorithm . . . 3
3.2 An example . . . 4
3.3 Turner's combinators . . . 4
3.4 Reading compiled code . . . 5

**4 From trees to graphs** **5**

**5 The reduction machine** **6**
5.1 Finding the next redex . . . 6
5.2 The rules . . . 7
5.3 Weak head normal form, and printing . . . 7

**6 Worked examples** **8**
6.1 A function call . . . 8
6.2 Sharing: twice . . . 8
6.3 Laziness . . . 8

**7 Conditionals and recursion** **9**
7.1 IF without thunks . . . 9
7.2 Recursion as a cycle . . . 9
7.3 Factorial . . . 9

**8 Laziness, for better and worse** **10**
8.1 Infinite lists . . . 10
8.2 Church numerals, as in the textbook . . . 11
8.3 Space leaks . . . 11

**9 An implementation on VM16** **11**
9.1 Representation . . . 12
9.2 The compiler and the graph format . . . 12
9.3 Definitions . . . 12
9.4 The reducer . . . 13
9.5 Garbage collection . . . 14

**10 Measurements** **14**

**11 Variations and descendants** **16**

**12 Summary** **17**

**Exercises** **17**

**13 Reference card** **18**

# 1 Introduction

The SECD machine [9] and the Categorical Abstract Machine (CAM) [2] evaluate $\lambda$-terms in much the same way. Both keep an *environment* that tells them the value of each variable, both build *closures* that pair a function's code with the environment it was defined in, and both evaluate an argument before passing it to a function. The first two are the price of having variables; the third is a choice.

In 1979 David Turner described a different way [14]. If a program contains no variables, it needs no environments and no closures. And any $\lambda$-term can be translated into a term without variables, built only from a few fixed functions called *combinators*. The idea goes back to Schönfinkel [13] and Curry [5], who showed in the 1920s and 1930s that two combinators, S and K, are enough to express every $\lambda$-term. Turner's contributions were a translation that produces reasonably small code, and an efficient way to run it: represent the program as a graph of application nodes and *rewrite* the graph, one combinator at a time, until a value is left.

Turner used the technique to implement his languages SASL and KRC, and later Miranda, the most direct ancestor of Haskell. It has three attractive properties.

1. **The machine is tiny.** Each combinator is a fixed rewrite rule, such as $\mathsf{S}\,f\,g\,x \;\to\; f\,x\,(g\,x)$. The whole machine is a loop that finds the next combinator to apply and rewrites the graph according to its rule. Cambridge researchers built it in hardware, as SKIM [4].
2. **Evaluation is lazy for free.** Rewriting a shared node in place means that every part of the program that refers to it sees the result. So an argument is evaluated at most once, and only if it is needed: call-by-need, or *lazy evaluation*, without any extra machinery.
3. **Recursion is a cycle.** The fixpoint combinator Y can be implemented by making a node point to itself, so a recursive function costs nothing per call to set up.

The price is that programs take many small steps, and that the translation can make them large. We will measure both.

**Plan.** Section 2 introduces combinators and section 3 the translation into them, bracket abstraction, with Turner's improvements. Section 4 explains why the program is a graph and not a tree, and section 5 gives the reduction machine. Section 6 traces programs through it, section 7 adds conditionals and recursion, and section 8 looks at laziness, for better and worse. Section 9 describes an implementation on the VM16 virtual machine and section 10 measures it. The paper ends with the descendants of the technique, exercises and a reference card.

**Notation.** We use the language of the companion tutorials: single-letter variables, $\lambda x\, y.M$ for $\lambda x.\lambda y.M$, application associating to the left, 15-bit integers, and curried operators written in prefix form, as in $-\ n\ 1$ for $n-1$. The implementation reads the same syntax, with \ for $\lambda$. It also accepts longer names, infix operators such as `n * fac (n-1)`, **if**–**then**–**else**, and a program made of named definitions (section 9.3).

# 2 Combinators

A *combinator* is a $\lambda$-term without free variables. A few of them are useful enough to have names:

| combinator | definition | rewrite rule |
|---|---|---|
| I | $\lambda x.x$ | $\mathsf{I}\,x \to x$ |
| K | $\lambda x\,y.x$ | $\mathsf{K}\,x\,y \to x$ |
| S | $\lambda f\,g\,x.f\,x\,(g\,x)$ | $\mathsf{S}\,f\,g\,x \to f\,x\,(g\,x)$ |

I is the identity, and K makes constant functions: $\mathsf{K}\,x$ is the function that ignores its argument and returns $x$. S is less obvious. It takes two functions and an argument, gives the argument to both, and applies the first result to the second. We will see in the next section that S is exactly what is needed to push an argument down into both halves of an application.

**Definition 1.** A *combinator term* is built from combinators, constants and application, with no $\lambda$s and no variables. A combinator applied to as many arguments as its rule needs is a *redex*, and replacing it by the right-hand side of its rule is a *reduction*.

Reduction never needs to substitute anything: the rules only rearrange their arguments. That is the point. For example, S K K behaves like I:

$$\mathsf{S}\,\mathsf{K}\,\mathsf{K}\,x \to \mathsf{K}\,x\,(\mathsf{K}\,x) \to x.$$

So I is not even needed; S and K suffice, a fact due to Schönfinkel [13]. A machine for combinator terms needs no environment, because there is nothing to look up, and no closures, because there is nothing to capture. It only has to find redexes and rewrite them.

# 3 Bracket abstraction

## 3.1 The basic algorithm

How do we get rid of the $\lambda$s? The trick is to eliminate one variable at a time. Write $[x]\,e$ for a term without $x$ such that

$$([x]\,e)\,x \to^{*} e.$$

$[x]\ e$ behaves like $\lambda x.e$, but it is built from combinators. If $e$ itself contains no $\lambda$s, three cases suffice:

$$\begin{aligned}
[x]\ x &= \mathsf{I} \\
[x]\ e &= \mathsf{K}\,e \qquad \text{if } e \text{ is a constant, a combinator or a variable other than } x \\
[x]\ (e_1\ e_2) &= \mathsf{S}\ ([x]\ e_1)\ ([x]\ e_2)
\end{aligned}$$

The third case is where S comes in. Applying the right-hand side to $x$ gives $([x]\ e_1)\ x\ (([x]\ e_2)\ x)$ by the rule for S, and each half then turns back into $e_1$ and $e_2$.

To compile a whole program, we work from the inside out: for each $\lambda x.M$, first compile $M$, which removes the $\lambda$s inside it, then abstract $x$ from the result.

**Theorem 1.** *If $e$ contains no $\lambda$s, then $[x]\ e$ contains no $x$, and $([x]\ e)\ x\ \to^*\ e$.*

*Proof.* By induction on $e$. For $x$, $\mathsf{I}\,x\ \to\ x$. For any other leaf $e$, $\mathsf{K}\,e\,x\ \to\ e$. For an application, $\mathsf{S}\ ([x]\ e_1)\ ([x]\ e_2)\ x\ \to\ ([x]\ e_1)\ x\ (([x]\ e_2)\ x)$, and by induction the two parts reduce to $e_1$ and $e_2$. None of the right-hand sides introduces $x$. □

### 3.2 An example

Take $\lambda x\ y.\ y\ x$, which applies its second argument to its first. The inner abstraction comes first:

$$[y]\ (y\ x) = \mathsf{S}\ ([y]\ y)\ ([y]\ x) = \mathsf{S}\ \mathsf{I}\ (\mathsf{K}\,x).$$

Now abstract $x$ from that:

$$\begin{aligned}
[x]\ (\mathsf{S}\ \mathsf{I}\ (\mathsf{K}\,x)) &= \mathsf{S}\ ([x]\ (\mathsf{S}\ \mathsf{I}))\ ([x]\ (\mathsf{K}\,x)) \\
&= \mathsf{S}\ (\mathsf{S}\ (\mathsf{K}\,\mathsf{S})\ (\mathsf{K}\,\mathsf{I}))\ (\mathsf{S}\ (\mathsf{K}\,\mathsf{K})\ \mathsf{I}).
\end{aligned}$$

That is correct, but it is a lot of code for such a small function: 9 application cells. Two improvements bring it down to 1.

### 3.3 Turner's combinators

The S rule passes $x$ to both halves of an application, even when only one half needs it. Turner [14, 15] added combinators for the cases where it does not, and simplified each S as it is built:

| simplification | combinator | meaning |
|---|---|---|
| $\mathsf{S}\ (\mathsf{K}\,p)\ (\mathsf{K}\,q) = \mathsf{K}\ (p\ q)$ | | no part needs $x$ |
| $\mathsf{S}\ (\mathsf{K}\,p)\ \mathsf{I} = p$ | | $\lambda x.\ p\ x$ is $p$ ($\eta$) |
| $\mathsf{S}\ (\mathsf{K}\,p)\ q = \mathsf{B}\ p\ q$ | $\mathsf{B}\ f\ g\ x\ \to\ f\ (g\ x)$ | only the argument needs $x$ |
| $\mathsf{S}\ p\ (\mathsf{K}\,q) = \mathsf{C}\ p\ q$ | $\mathsf{C}\ f\ g\ x\ \to\ f\ x\ g$ | only the function needs $x$ |
| $\mathsf{S}\ (\mathsf{K}\,p)\ (\mathsf{B}\ q\ r) = \mathsf{B}^*\ p\ q\ r$ | $\mathsf{B}^*\ k\ f\ g\ x\ \to\ k\ (f\ (g\ x))$ | B inside B |
| $\mathsf{S}\ (\mathsf{B}\ p\ q)\ (\mathsf{K}\,r) = \mathsf{C}'\ p\ q\ r$ | $\mathsf{C}'\ k\ f\ g\ x\ \to\ k\ (f\ x)\ g$ | C with a function $p$ applied first |
| $\mathsf{S}\ (\mathsf{B}\ p\ q)\ r = \mathsf{S}'\ p\ q\ r$ | $\mathsf{S}'\ k\ f\ g\ x\ \to\ k\ (f\ x)\ (g\ x)$ | S with a function $p$ applied first |

The rules are tried in this order. The first is really the K case of the basic algorithm, applied to whole subterms: if neither half contains $x$, the whole application does not either. B and C are the two halves of S: B routes $x$ only into the argument, C only into the function. The *primed* combinators $\mathsf{S}'$, $\mathsf{B}^*$ and $\mathsf{C}'$ deal with something that happens all the time in curried code: a function of several arguments, such as $\mathsf{S}\ (\mathsf{B}\ p\ q)\ r$, where the first combinator only exists to apply $p$ to the result. They do the same job with one cell fewer and one reduction fewer.

| $\lambda$-term | with S, K, I only | with Turner's rules |
|---|---|---|
| $\lambda x\, y.\, x$ | S (K K) I | K |
| $\lambda x\, y.\, y\, x$ | S (K (S I)) (S (K K) I) | C I |
| $\lambda x.\, +\, x\, 1$ | S (S (K +) I) (K 1) | C + 1 |
| $\lambda x.\, x\, x$ | S I I | S I I |
| $\lambda f\, x.\, f\, (f\, x)$ | S (S (K S) (S (K K) I)) (S (S (K S) (S (K K) I)) (K I)) | S B I |

**Table 1:** Bracket abstraction with and without Turner's rules. The plain algorithm here already includes the rule S (K $p$) (K $q$) = K ($p$ $q$), which is why $[x]$ (S I) in the second line became K (S I).

With these rules, the example becomes:

$$[y]\ (y\ x) = \mathsf{S}\ \mathsf{I}\ (\mathsf{K}\, x) = \mathsf{C}\ \mathsf{I}\ x$$
$$[x]\ (\mathsf{C}\ \mathsf{I}\ x) = \mathsf{S}\ (\mathsf{K}\ (\mathsf{C}\ \mathsf{I}))\ \mathsf{I} = \mathsf{C}\ \mathsf{I}.$$

One cell. And indeed $\mathsf{C}\ \mathsf{I}\ a\ f \ \to\ \mathsf{I}\ f\ a \ \to\ f\ a$.

Table 1 shows more examples, as printed by the implementation's compiler, `lam2graph.py -t`.

**How large can the code get?**

Each abstraction can make a term up to three times larger, and nested abstractions compound: with the three basic rules alone, code can grow exponentially with the nesting depth of $\lambda$s. The K simplification avoids that, but the worst case remains quadratic in the size of the program, even with Turner's rules. Algorithms with a linear worst case exist; they use whole families of combinators, such as $\mathsf{B}_n$ and $\mathsf{C}_n$ for routing an argument past $n$ others. Kiselyov [8] gives a recent and very readable account. For the programs in this tutorial, Turner's rules shrink the code by a factor of 2 to 4 (table 5).

### 3.4 Reading compiled code

Factorial is

$$\textbf{rec}\ f.\ \lambda n.\ \textbf{if}\ (=\ n\ 0)\ 1\ (*\ n\ (f\ (-\ n\ 1)))$$

where **rec** $f.M$ is Y ($\lambda f.M$) (section 7) and **if** $c\ t\ e$ is the combinator IF applied to $c$, $t$ and $e$. It compiles to a term of 16 cells:

$$\mathsf{Y}\ (\mathsf{B}^*\ (\mathsf{S}\ (\mathsf{C}'\ \mathsf{IF}\ (\mathsf{C} = 0)\ 1))\ (\mathsf{S}\ *)\ (\mathsf{C}\ \mathsf{B}\ (\mathsf{C} - 1))).$$

It can be read piece by piece. C = 0 is $\lambda n.\, =\, n\ 0$, and C − 1 is $\lambda n.\, -\, n\ 1$. C B (C − 1) takes $f$ and then $n$ and gives $f\ (-\ n\ 1)$; S $*$ multiplies $n$ by that; and C′ IF (C = 0) 1 takes the result as the else-branch and builds the conditional. $\mathsf{B}^*$ and S distribute $f$ and $n$ to the parts that need them. Nobody would write it that way, but it needs no variables.

## 4 From trees to graphs

A combinator term is a binary tree of applications. Each application is a cell with two fields, the function and the argument; the leaves are combinators and numbers. To reduce S $f\ g\ x$, we could build the tree for $f\ x\ (g\ x)$, with two copies of $x$. But $x$ may be a large unevaluated expression, and copying it would mean evaluating it twice later.

The solution is not to copy it: both new cells point to the same $x$ (fig. 1). From then on the program is a *graph*, in which a cell can have several parents. This is safe as long as nothing ever changes the *meaning* of a cell, and the rewrite rules never do. They only replace a cell's contents by something equal to it.

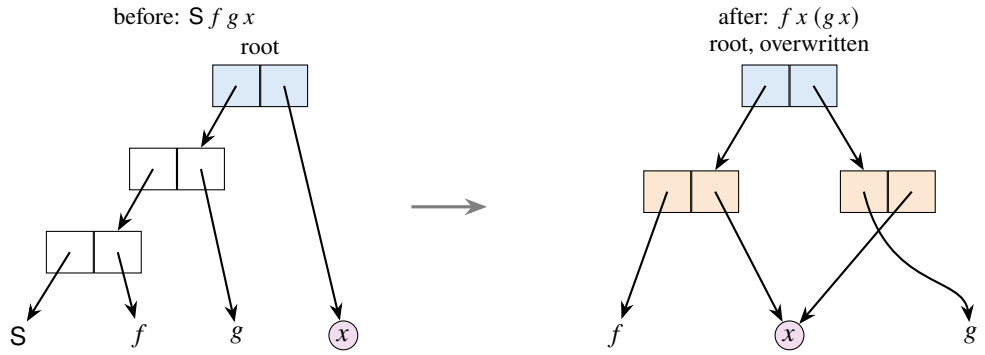



**Figure 1:** The S rule on a graph. Each box pair is an application cell: the left half points to the function, the right half to the argument. The root of the redex (blue) is overwritten; two new cells (orange) are allocated; and $x$ is shared rather than copied. The three old cells of the spine become garbage unless something else refers to them.

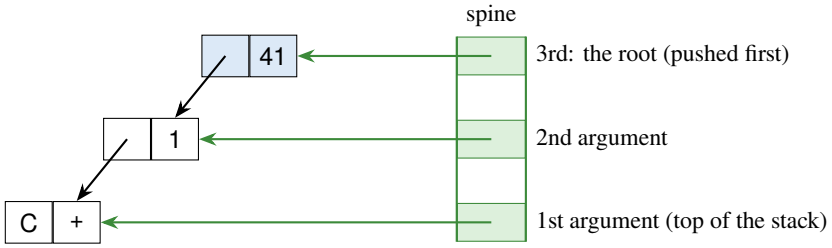



**Figure 2:** Unwinding C + 1 41. The machine follows function fields from the root down to the combinator C, pushing each cell. The argument of the $i$th cell from the top is the $i$th argument of C, and the 3rd cell is the root of the redex, because C takes three arguments.

**Overwrite the root.** Notice what happened to the root of the redex, the outermost cell of S $f$ $g$ $x$. The rule does not build a new cell for $f\ x\ (g\ x)$ and leave the old one alone. It *overwrites* the root with the new contents. Other parts of the program may point to that same cell, because it may itself be a shared argument. When it is overwritten, they all see the reduced version, and none of them will reduce it again. This is what makes evaluation *lazy* in the full sense, call-by-need: an expression is reduced only when something needs its value, and at most once.

**Indirections.** Sometimes the result is not an application. After K $x$ $y$ $\longrightarrow$ $x$, the root should become $x$ itself, but if $x$ is a number or a combinator, it does not fit in a cell, and if $x$ is a cell, copying its two fields into the root would duplicate it, losing sharing. The standard answer is to overwrite the root with I $x$, an *indirection*. Anything that reaches the root then finds I $x$ and goes on to $x$. In this machine an indirection is not a special kind of cell: it is just an ordinary I redex, and reducing it is an ordinary reduction.

# 5 The reduction machine

## 5.1 Finding the next redex

Lazy evaluation reduces the *leftmost outermost* redex first: the one at the head of the whole expression. To find it, the machine walks down the left-hand, *function* fields from the root, remembering every cell it passes, until it reaches a leaf. This path is the *spine*, and the walk is called *unwinding*. The cells of the spine are kept on a stack, with the innermost on top. Figure 2 shows the spine of C + 1 41.

When the walk ends at a leaf, there are three cases:

- **A combinator with enough arguments.** If the combinator needs $n$ arguments and the spine has at least $n$ cells, the top $n$ cells hold the arguments, in their argument fields, and the $n$th is the root of

| rule | new cells | what the root becomes |
|---|---|---|
| $\mathsf{I}\,x \;\to\; x$ | 0 | (the parent is pointed at $x$ directly) |
| $\mathsf{K}\,x\,y \;\to\; x$ | 0 | $\mathsf{I}\,x$ |
| $\mathsf{S}\,f\,g\,x \;\to\; f\,x\,(g\,x)$ | 2 | $(f\,x)\;(g\,x)$ |
| $\mathsf{B}\,f\,g\,x \;\to\; f\,(g\,x)$ | 1 | $f\;(g\,x)$ |
| $\mathsf{C}\,f\,g\,x \;\to\; f\,x\,g$ | 1 | $(f\,x)\;g$ |
| $\mathsf{S}'\,k\,f\,g\,x \;\to\; k\,(f\,x)\,(g\,x)$ | 3 | $(k\,(f\,x))\;(g\,x)$ |
| $\mathsf{B}^*\,k\,f\,g\,x \;\to\; k\,(f\,(g\,x))$ | 2 | $k\;(f\,(g\,x))$ |
| $\mathsf{C}'\,k\,f\,g\,x \;\to\; k\,(f\,x)\,g$ | 2 | $(k\,(f\,x))\;g$ |
| $\mathsf{Y}\,f \;\to\; f\,(\mathsf{Y}\,f)$ | 0 | $f$ root: the root points to itself |
| $\mathsf{IF}\,c\,t\,e \;\to\; t$ or $e$ | 0 | $\mathsf{I}\,t$ if $c \neq 0$, else $\mathsf{I}\,e$, after reducing $c$ |
| $+\,a\,b \;\to\; a + b$ | 0 | $\mathsf{I}\,(a + b)$, after reducing $a$ and $b$; likewise $- * / \,\%\, = <$ |

**Figure 3:** The rewrite rules. "New cells" counts the cells allocated; the root of the redex is always reused. IF and the operators reduce some arguments first, by a recursive call of the machine.

the redex. The machine applies the rule, overwrites the root, pops the $n$ cells, and unwinds again from the root.

- **A combinator with too few arguments.** Then the expression is a partial application such as + 3. It cannot be reduced further: it is in *weak head normal form* (WHNF), and the machine stops.
- **A number.** If the spine is empty, the number is the result. If not, the program tried to apply a number to an argument, which is an error.

Note that after a rewrite the machine continues from the root of the redex, not from the top of the whole program. The cells below the root on the stack are still on the spine, and nothing above them has changed.

## 5.2 The rules

Figure 3 lists every rule. The combinator rules only rearrange pointers; the rules for IF and the arithmetic operators are different, because they need the *values* of some arguments. To get the value of an argument $a$, the machine runs itself recursively on $a$, with a new spine on top of the old one, until $a$ is in WHNF. Because the graph is updated in place, the argument cell now holds its value (or an indirection to it), for the benefit of anyone else who refers to it.

**Short-circuiting indirections.** When the machine reduces $\mathsf{I}\,x$, it also changes the parent cell on the spine, if there is one, to point at $x$ instead of at the indirection. This is free (one store), and it means an indirection is usually passed through only once. The same happens after K, IF and the operators, which leave an indirection behind in the root: the parent is pointed straight at the result.

## 5.3 Weak head normal form, and printing

The machine stops at WHNF: a number, or a function still waiting for arguments. It does not reduce inside a partial application, so $\lambda x.\,+\;1\;2$ is not simplified to a constant function of 3 until it is applied. That is the same choice the SECD machine and the CAM make when they stop at a closure, and for the same reason: the inside of a function only matters when it is called. The implementation prints a number as a number and anything else as `<function>`.

# 6 Worked examples

The traces in this section were produced by `prototype.py --trace`, a Python model of the implementation that makes exactly the same reductions in the same order. Each line shows the whole program graph after one more reduction, written as a term, so a shared cell appears once for each of its parents. Indirections are shown as I $x$.

## 6.1 A function call

$(\lambda x.\ +\ x\ 1)\ 41$ compiles to C + 1 41 (table 1), and runs in two reductions:

| step | rule | graph |
|---|---|---|
| 0 | | C + 1 41 |
| 1 | C | + 41 1 |
| 2 | + | I 42 |

After step 1, unwinding the root finds + with two arguments. The machine evaluates 41 and 1 (both are already numbers) and overwrites the root with I 42; with no parent to update, the machine continues at 42, an empty spine and a number, so it stops. The CAM needed 13 instructions for the same program.

## 6.2 Sharing: twice

$\lambda f\, x.\ f\ (f\ x)$ compiles to S B I. Applied to + 1 and 0, it shows the first sharing:

| step | rule | graph |
|---|---|---|
| 0 | | S B I (+ 1) 0 |
| 1 | S | B (+ 1) (I (+ 1)) 0 |
| 2 | B | + 1 (I (+ 1) 0) |
| 3 | I | + 1 (+ 1 0) |
| 4 | + | + 1 (I 1) |
| 5 | + | I 2 |

At step 1, S gives its third argument, + 1, to both B and I. The two occurrences in the line are the same cell. At step 4, the outer + needs the value of its second argument, so the machine calls itself on the cell + 1 0, which is overwritten with I 1. Step 3 was an I reduction that the compiler put there: S B I uses I to say "and pass $f$ unchanged".

## 6.3 Laziness

$(\lambda x\ y.\ y)\ (/\ 1\ 0)\ 5$ compiles to K I (/ 1 0) 5. K discards its second argument, so the division by zero is never attempted:

| step | rule | graph |
|---|---|---|
| 0 | | K I (/ 1 0) 5 |
| 1 | K | I 5 |
| 2 | I | I 5 |

After step 1, the root of the K redex is the cell K I (/ 1 0), which becomes I I; its parent, the program root, is pointed straight at I, so the program reads I 5. Step 2 reduces that, and the machine stops at 5. On the SECD machine and the CAM, the same program fails with "division by zero".




**Figure 4:** The Y rule. The root is overwritten with $h$ applied to the root itself. Unfolding the recursion never copies anything.

# 7 Conditionals and recursion

## 7.1 IF without thunks

The companion tutorials spend some time on conditionals. In a strict machine, *cond c t e* written as an ordinary function evaluates both $t$ and $e$ before choosing one, so a recursive function such as factorial never stops. The SECD tutorial wraps each branch in $\lambda d.$ and applies the chosen one to a dummy argument, and both machines also get a special **if** form that evaluates only one branch.

In a lazy machine none of this is needed. IF is an ordinary combinator of three arguments, and the rule for it reduces only the condition. The branch it does not choose is never touched. Even Church booleans, $\lambda x\, y.\, x$ and $\lambda x\, y.\, y$, work as conditionals: they compile to K and K I, and only the chosen argument is ever reduced. IF is just faster.

## 7.2 Recursion as a cycle

**rec** $f.M$ means $M$, where $f$ refers to $M$ itself. The compiler turns it into Y $(\lambda f.M)$, where Y is the fixpoint combinator, Y $h = h$ (Y $h$). As a $\lambda$-term, Y has to make a copy of $h$ (Y $h$) each time the recursion unfolds. In a graph, it can do better. The rule overwrites the root, the cell Y $h$, with $h$ applied to *the root itself* (fig. 4). The result is a cycle in the graph: the function's own definition is reachable from inside it. Every recursive call then follows the cycle back, and no further copying is ever needed. A recursive definition costs one reduction for the whole run.

**rec does not need a function.** Since nothing about the cycle requires $M$ to be a function, **rec** $l.\, \lambda c\, n.\, c\ 1\ l$ is an infinite list of ones (lists are introduced in section 8), represented by a single cycle. The CAM's **rec** must be applied to a $\lambda$; here it can define any value in terms of itself. A definition with no content, **rec** $x.\, x$, becomes Y I, then a cell that is an indirection to itself; the implementation recognises that case and reports "a value defined as itself" instead of looping.

## 7.3 Factorial

Table 2 traces factorial applied to 1. Step 1 builds the cycle, shown as @ where the line refers back to the cell being printed. Steps 2 to 7 distribute $n$ and $f$, test $n = 0$, and choose the else-branch. Steps 8 to 12 do the same for the recursive call.

Two things are worth noticing. First, the argument of the recursive call, C − 1 1, which is $n - 1$, appears twice in line 12: once in the test and once in the else-branch. They are the same cell. At step 14 it is rewritten to − 1 1, and both occurrences change; at step 15 it is reduced to I 0, and both see the value. Without sharing, $n - 1$ would be computed twice in every call.

Second, steps 7 and 17 show how the machine handles a conditional in tail position: IF replaces its root with an indirection to the chosen branch and continues there, with no recursive call of the machine. Only the multiplication, which needs the value of the recursive call, nests. That is why factorial of $n$ nests $n$ levels deep, while a loop whose recursive call is the whole result runs in constant stack space (section 8.3).

| step | rule | graph (with $F$ = S (C′ IF (C = 0) 1)) |
|---|---|---|
| 0 | | Y (B* $F$ (S ∗) (C B (C − 1))) 1 |
| 1 | Y | B* $F$ (S ∗) (C B (C − 1)) @ 1 |
| 2 | B* | $F$ (S ∗ (C B (C − 1) @)) 1 |
| 3 | S | C′ IF (C = 0) 1 1 (S ∗ (C B (C − 1) …) 1) |
| 4 | C′ | IF (C = 0 1) 1 (S ∗ … 1) |
| 5 | C | IF (= 1 0) 1 (S ∗ … 1) |
| 6 | = | IF (I 0) 1 (S ∗ … 1) |
| 7 | IF | I (S ∗ (C B (C − 1) …) 1) |
| 8 | S | I (∗ 1 (C B (C − 1) … 1)) |
| 9 | C | I (∗ 1 (B … (C − 1) 1)) |
| 10 | B | I (∗ 1 ($F$ … (C − 1 1))) |
| 11 | S | I (∗ 1 (C′ IF (C = 0) 1 (C − 1 1) (… (C − 1 1)))) |
| 12 | C′ | I (∗ 1 (IF (C = 0 (C − 1 1)) 1 (… (C − 1 1)))) |
| 13 | C | I (∗ 1 (IF (= (C − 1 1) 0) 1 (… (C − 1 1)))) |
| 14 | C | I (∗ 1 (IF (= (− 1 1) 0) 1 (… (− 1 1)))) |
| 15 | − | I (∗ 1 (IF (= (I 0) 0) 1 (… (I 0)))) |
| 16 | = | I (∗ 1 (IF (I 1) 1 (… (I 0)))) |
| 17 | IF | I (∗ 1 (I 1)) |
| 18 | ∗ | I (I 1) |

**Table 2:** (**rec** $f\,.\,\lambda n.$ **if** $(=n\,0)$ 1 $(*\,n\,(f\,(-n\,1))))$ 1, abridged from `prototype.py --trace`: "…" stands for parts of the cyclic function that do not change. Steps 13 to 15 show sharing: the argument $n-1$ of the recursive call is one cell, used both by the test and by the multiplication, so it is computed once.

Factorial of 7 takes 90 reductions, against 171 instructions on the CAM (87 if *fac* calls itself by name, as in section 9.3).

# 8 Laziness, for better and worse

## 8.1 Infinite lists

Lazy evaluation makes it possible to define infinite data and use only the part that is needed. Our language has no built-in lists, but they are easy to encode as functions. A list is a function of two arguments, what to do with a non-empty list and what to give for the empty one:

$$cons\ h\ t = \lambda c\,n.\,c\ h\ t \qquad\qquad nil = \lambda c\,n.\,n.$$

To use a list $l$, apply it to a function of the head and tail, and to a default: $l\ (\lambda h\,t.\ \ldots)\ d$. With these, the integers from $n$ upwards are

$$from\ n = cons\ n\ (from\ (+\ n\ 1)),$$

an infinite list. A strict machine would loop forever building it. Here $cons\ n\ (f\ (+\,n\,1))$ is in WHNF as soon as *cons* has its two arguments: the tail is a graph that has not been reduced, and will only be reduced if someone asks for it.

The program `primes.lam` builds on this. It filters multiples out of *from* 2 with the sieve of Eratosthenes, getting the infinite list of primes, and takes element 99. It is written as a series of definitions, each ended by a semicolon, followed by the expression to evaluate:

```
cons h t c n = c h t;
nil c n = n;

from n = cons n (from (+ n 1));
filter p l = l (\h t. if (p h) (cons h (filter p t)) (filter p t)) nil;
sieve l = l (\h t. cons h (sieve (filter (\x. % x h) t))) nil;
nth k l = l (\h t. if (= k 0) h (nth (- k 1) t)) 0;
```

```
primes = sieve (from 2);

nth 99 primes                          # counting from 0
```

It prints 541 after 154,682 reductions. Only the first 541 integers and the first 100 primes are ever built. Interestingly, the deepest nesting of the machine during the run is only 4: almost all of the work is done by IF choosing a branch and by combinators rearranging pointers, which never nest.

### 8.2 Church numerals, as in the textbook

`church.lam` computes 5! with Church numerals, Church booleans and the fixpoint combinator Y written as a $\lambda$-term, $\lambda f.(\lambda x.f\ (x\,x))\ (\lambda x.f\ (x\,x))$, exactly as a textbook would write it:

$$\mathsf{Y}\ (\lambda f\, n.\ \mathit{iszero}\ n\ \mathit{one}\ (\mathit{times}\ n\ (f\ (\mathit{pred}\ n)))).$$

Neither the SECD machine nor the CAM can run it: they evaluate the argument $x\,x$ inside Y, and both branches of the conditional, before anything else, and recurse forever. (The SECD version, `fac.lam`, has to use the call-by-value fixpoint combinator $Z$ and wrap each branch in a $\lambda d$.) Here it gives 120 after 16,918 reductions.

### 8.3 Space leaks

The flip side of laziness is that unevaluated expressions are kept until they are needed, and they can pile up. A classic example is a loop with an accumulator:

$$\mathbf{rec}\ f.\,\lambda n\,a.\ \mathbf{if}\ (=\,n\ 0)\ a\ (f\ (-\,n\ 1)\ (+\,a\ 1))$$

applied to 3000 and 0. The test $=\,n\ 0$ forces $n$ at every step, but nothing forces $a$. After four rounds the accumulator is the graph

$$+\ (+\ (+\ (+\ 0\ 1)\ 1)\ 1)\ 1,$$

and after 3000 it is a chain of 3000 unevaluated additions. Evaluating it at the end needs the machine to nest 3000 levels deep, because each + needs the value of the one inside it, and the implementation runs out of stack. The cure is to force the accumulator on every round, for example by testing it: with $\mathbf{if}\ (<\,a\ 0)\ 0\ (f\ \ldots)$ around the recursive call, the same loop runs in constant space, in 63,008 reductions. Haskell provides `seq` for exactly this purpose (exercise 5).

## 9 An implementation on VM16

VM16 is a small 16-bit virtual machine with sixteen registers, a data stack and a return stack, designed to be implemented in a few kilobytes on 8-bit processors. Its C interpreter is the reference, and a Z80 interpreter runs the same images. The implementation is in two parts:

- `lam2graph.py`, a compiler in Python, reads a program and writes the compiled graph to a text file;
- `graph.asm`, a VM16 program of 2,451 bytes, reads that file, reduces the graph, and prints the result followed by the number of reductions and garbage collections.

The script `run` does both steps:

```
$ python3 lam2graph.py fac.lam         # writes fac.graph
$ ../vm16 graph.v16 < fac.graph
5040
[87 reductions, 0 collections]
$ ./run fac.lam                        # the same
```

Splitting the work this way keeps the part that runs on the small machine to just the reducer. The compiler could later be written in the language itself and compiled to a graph, as Turner did for SASL.

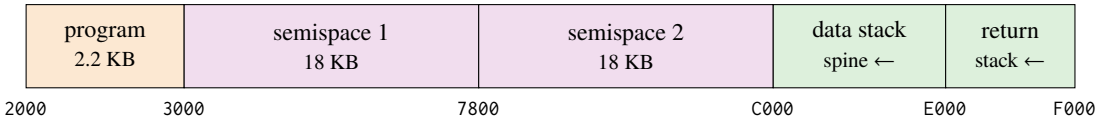



**Figure 5:** Memory layout of `graph.asm`, not to scale. The heap is two semispaces of 4,608 cells each. The spine grows down from `0xE000`; the machine stops with "out of stack" before it reaches the heap.

## 9.1 Representation

Every heap object is an application cell of two 16-bit words, the function and the argument. There is no tag. Every other value fits into one word, and the kind of a word can be told from the word alone:

| word | meaning | example |
|---|---|---|
| odd | the number $n$, stored as $2n + 1$ | 41 is `0x0053` |
| even, below `0x100` | a combinator | K is 4, S is 6, + is 22 |
| even, from `0x3000` | a pointer to a cell | |

Combinator codes are even and small so that they can be used directly as byte offsets into tables of two-byte entries: the address of each rule, its number of arguments, and its name.

Compared with the CAM, a cell is two words instead of three, because there is only one kind of cell. Memory is laid out as in fig. 5.

## 9.2 The compiler and the graph format

`lam2graph.py` reads the program into a syntax tree and then works through it recursively:

- A leaf is left alone.
- An application is compiled in place: its two halves are compiled and stored back into the same cell.
- A $\lambda$-cell $(\lambda x, M)$ is replaced by $[x]\, M'$, where $M'$ is the compiled body. The abstraction is a recursive procedure that follows the three basic rules, with each S built by a procedure that applies Turner's simplifications in the order of section 3.3.

Finally the compiled term is checked for variables: any variable left over was not bound by a $\lambda$, and unless it names a definition (section 9.3) it is reported as unbound. Turner's rules need only a look at the first two levels of their arguments, to recognise K $p$, I, and B $p\ q$, so the compiler is short: about 40 lines of Python, or about 150 instructions in an earlier version written in VM16 assembly.

The compiled graph is written in postfix: a combinator or a number is a token, and `f a @` is $f$ applied to $a$. This is the factorial of section 3.3, written with **rec**, applied to 7:

```
Y B* S C' IF @ C = @ 0 @ @ 1 @ @ @ S * @ @ C B @ C - @ 1 @ @ @ @ 7 @
```

`graph.asm` loads it with the data stack: each token pushes a word, and `@` pops two words and pushes a new cell made of them. At the end of the input exactly one word must be left, the root of the graph.

## 9.3 Definitions

A program can also be a series of definitions, each ended by a semicolon, followed by the expression to evaluate (or, without one, a definition named *main*). A definition $f\ x\ y = M$ means $f = \lambda x\, y.M$, and names may be longer than one letter:

```
iszero n = = n 0;
fac n = if (iszero n) 1 (* n (fac (- n 1)));
fac 7
```

One could treat this as one big term, ($\lambda$ *iszero fac*. ...) ..., with Y for the recursion, but then bracket abstraction would have to pass every name down to every place that uses it. Turner's implementations did something simpler, which a graph makes easy: each definition is compiled on its own, with the names of the other definitions left as leaves, and each name then becomes a pointer to its definition's cell. A recursive definition is a cycle, as Y makes, and needs neither Y nor **rec**; mutual recursion is just as easy. For the program above, `lam2graph.py -t` prints

```
fac = S (C' IF iszero 1) (S * (B fac (C - 1)))
iszero = C = 0
fac 7
```

where *fac* and *iszero* inside the terms are pointers. Written this way, `church.lam` takes 52 cells instead of 98, and `primes.lam` 65 instead of 107, plus one cell per definition.

In the graph file, `%`*n* at the start makes *n* cells for the definitions, `$`*k* pushes the cell of definition *k*, and `=`*k* pops a cell and copies its two words into the cell of definition *k*. Copying, rather than storing a pointer, means that every reference to the definition, made before or after, already points at the definition itself and not at an indirection. Only the definitions that the program can reach are written. The loader is about 160 instructions, including the input routines it shares with the rest of the program.

`lam2graph.py` also reads programs in MORC, a syntax with the usual infix operators and precedences, **if**–**then**–**else**, and lists built with `:` and `nil`:

```
fn fac n = if n = 0 then 1 else n * fac (n - 1);
fn main = fac 7;
```

It is only a different reader: `n * fac (n - 1)` becomes $*\ n\ (fac\ (-\ n\ 1))$, and the program compiles to the same combinators as the one above.

## 9.4 The reducer

The reducer is a procedure, `whnf`, that reduces the graph at a given cell to WHNF. Its spine is the VM16 data stack, and register `r9` remembers where the current spine began, so that nested calls (for IF and the operators) stack their spines on top of each other. Unwinding is a short loop:

```
unwind: LI      r1, 1
        AND     r1, r0
        BNZ     r1, wh_number           ; an odd word: a number
        BLTU    r0, r14, wh_comb        ; below the heap: a combinator
        PUSH    r0                      ; a cell: push it on the spine,
        GETDSP  r1
        BLTU    r1, r13, wh_full        ; (unless the stack is full)
        LW      r0, [r0]                ; and go down its function half
        JMP     unwind
```

At a combinator, the reducer compares the number of cells on the spine with the combinator's number of arguments, from a table, and either stops (a partial application) or jumps to the rule through another table. Each rule reads its arguments from the argument halves of the spine cells. This is the S rule:

```
r_s:    BGEU    r11, r10, rs_go         ; S f g x = f x (g x)
        CALL    collect                 ; (make room for 3 cells first)
rs_go:  GETDSP  r7
        LW      r1, [r7]
        LW      r1, [r1+2]              ; f
        LW      r2, [r7+2]
        LW      r2, [r2+2]              ; g
        LW      r0, [r7+4]              ; the root
        LW      r3, [r0+2]              ; x
```

```
        SW      r1, [r10]               ; a new cell (f x)
        SW      r3, [r10+2]
        SW      r10, [r0]               ; overwrite the root's function
        ADDI    r10, CELL
        SW      r2, [r10]               ; a new cell (g x)
        SW      r3, [r10+2]
        SW      r10, [r0+2]             ; overwrite the root's argument
        ADDI    r10, CELL
        ADDI    r7, 6                   ; pop the three spine cells
        SETDSP  r7
        JMP     unwind                  ; and unwind from the root
```

Register `r10` is the allocation pointer, and `r11` the end of the semispace minus three cells, so one comparison decides whether a rule can allocate without collecting. The Y rule is four instructions:

```
r_y:    POP     r0                      ; the root, Y f
        LW      r1, [r0+2]              ; f
        SW      r1, [r0]                ; the root becomes f ...
        SW      r0, [r0+2]              ; ... applied to the root itself
        JMP     unwind
```

The reducer, its rules and the arithmetic take about 940 bytes.

### 9.5 Garbage collection

The heap is managed by Cheney's copying collector [3], which is kept, with input and printing, in a separate file `lib.asm`. When the current semispace is full, every cell reachable from the *roots* is copied to the other semispace, and the two swap roles. Finding the roots is easy here, because the data stack holds every live pointer:

- **The spine is the root set.** Everything the program still needs is reachable from some cell on some spine, including the spines of suspended outer calls of `whnf`. The data stack also holds a few words that are not cells: saved spine bases, which are stack addresses, numbers, and rule addresses for the operators. None of them lies inside the heap, so the collector can safely leave them alone.
- **Room before reading.** A rule that allocates first checks that three cells are free, collecting if not, and only then reads its arguments from the spine. So no rule ever holds a heap pointer in a register across a collection, which would leave it pointing at the old copy.
- **Reloading after nesting.** IF and the operators call `whnf` on an argument, during which a collection may move everything. Afterwards they read the spine again rather than relying on registers.

The forwarding address of a copied cell is marked by storing 0, which is not a valid combinator, in its function half. Cycles made by Y need no special care: Cheney's algorithm copies each cell exactly once.

## 10 Measurements

Table 3 compares the graph reducer with the SECD machine and the CAM of the companion tutorials, both also written in VM16 assembly. The measure is the number of VM16 instructions executed, counted by the C interpreter, which is independent of the host. It is split in two, because the machines do different amounts of work before they start: the SECD machine and the CAM read and compile the program on VM16, while the graph reducer only loads a graph compiled in Python.

| program | machine | machine steps | GCs | VM16 instructions setup | VM16 instructions run |
|---|---|---|---|---|---|
| *fac* 7 | CAM, **rec** | 171 instructions | 0 | 6,998 | 3,133 |
| | graph, by name | 87 reductions | 0 | 2,725 | 6,509 |
| *fac* 7, with *Z* | SECD | 322 transitions | 0 | 8,791 | 20,947 |
| | CAM | 346 instructions | 0 | 11,040 | 6,555 |
| | graph | 151 reductions | 0 | 4,010 | 9,865 |
| *fib*(20) | SECD, with *Z* | 45,423 transitions | 702 | 4,297 | 61,157,464 |
| | CAM, **rec** | 448,762 instructions | 16 | 5,732 | 7,029,349 |
| | graph, by name | 240,795 reductions | 45 | 3,420 | 16,796,731 |
| | VM16 assembly | — | — | — | 175,169 |
| 5!, Church numerals | SECD | 52,044 transitions | 49 | 20,175 | 4,065,860 |
| | CAM | 73,681 instructions | 10 | 28,259 | 1,476,892 |
| | graph | 16,987 reductions | 2 | 14,384 | 914,239 |
| 5!, Church, with Y | graph only | 16,918 reductions | 2 | 9,616 | 910,174 |
| 100th prime | graph only | 154,682 reductions | 43 | 11,288 | 10,692,216 |

**Table 3:** The same programs on the three machines. *Setup* is everything before the machine's first step: for the SECD machine and the CAM, reading the source text and compiling it on VM16; for the graph reducer, only loading the graph that `lam2graph.py` compiled. *Run* is the rest, including printing the result, and is the fair comparison. The Church numeral program is `fac.lam` from the SECD tutorial, with its *Z* combinator and thunks, so that all three machines can run it; the last two rows are `church.lam` and `primes.lam`. On the graph reducer, *fac* and *fib* are `fac.lam` and `fib.lam`, which call themselves by name (section 9.3).

**First-order programs.** On *fac* and *fib*, the graph reducer takes fewer steps than the CAM, but each step costs more: about 70 VM16 instructions per reduction against 16 per CAM instruction. On *fib*(20) that makes it 2.4 times slower than the CAM, though still 3.6 times faster than the SECD machine. On the short runs of *fac* it is 1.5 to 2.1 times slower. Table 4 shows where the reductions go. Each call of *fib* takes 11 reductions, and only three of them do arithmetic; the rest move *n* to where it is needed. C and I alone are 36 % of the work.

The 70 instructions per reduction break down roughly as follows. Unwinding pushes 2.5 cells per reduction on average, at 9 instructions each, about 25 in all; checking the number of arguments and dispatching through the table takes 13; and the rule itself takes the rest, between 4 for Y and about 30 for the operators, which also call `whnf` on their arguments.

**Higher-order programs.** On Church numerals the picture reverses. The graph reducer needs 16,987 reductions where the CAM needs 73,681 instructions, and it is 1.6 times faster than the CAM and 4.4 times faster than the SECD machine. Church numerals are functions that apply other functions many times. The CAM and the SECD machine build a closure and an environment for every application, while a combinator rule only rearranges the pointers it was given. Sharing helps too: a numeral that is used several times is built once.

**Turner's combinators.** Table 5 shows what the extra combinators of section 3.3 are worth. They make the code 2 to 4 times smaller, and the programs 2.1 to 13 times faster, the most on the biggest program, where the plain S rule passes arguments down long paths that never use them.

**On a Z80.** The Z80 VM16 interpreter averages about 288 clock cycles per VM16 instruction on this program. At 4 MHz, *fac* 7 takes 0.68 seconds and the Church factorial 69 seconds, both measured on a Z80 emulator, and *fib*(20) would take about 20 minutes.

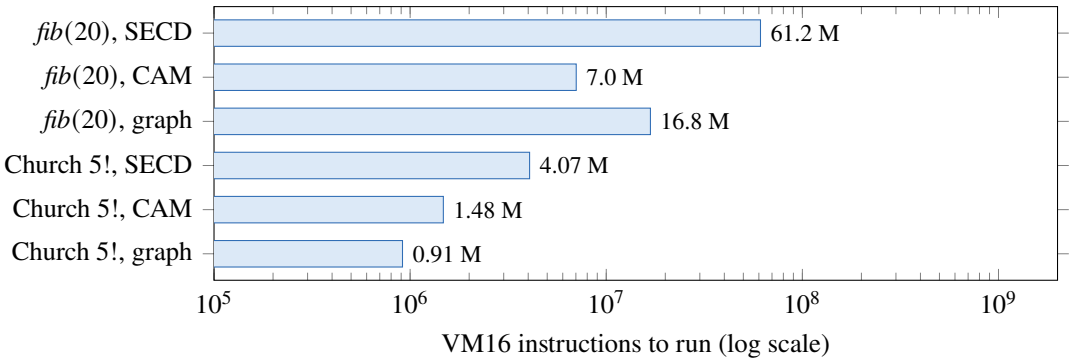



**Figure 6:** Two kinds of program. On *fib*, which is first-order arithmetic, the CAM is 2.4 times faster than the graph reducer. On Church numerals, which are nothing but higher-order functions, the graph reducer is the fastest, 1.6 times faster than the CAM.

| rule | reductions | per call | share |
|---|---|---|---|
| C | 43,781 | 2 | 18.2 % |
| I | 43,780 | 2 | 18.2 % |
| S | 32,836 | 1.5 | 13.6 % |
| S′ | 21,891 | 1 | 9.1 % |
| IF | 21,891 | 1 | 9.1 % |
| < | 21,891 | 1 | 9.1 % |
| − | 21,890 | 1 | 9.1 % |
| B | 10,945 | 0.5 | 4.5 % |
| B* | 10,945 | 0.5 | 4.5 % |
| + | 10,945 | 0.5 | 4.5 % |
| total | 240,795 | 11 | |

**Table 4:** The reductions in *fib*(20), which makes 21,891 calls, from `prototype.py --counts`. Half the calls are leaves ($n < 2$), which do not add.

## 11 Variations and descendants

**Hardware.**
SKIM, the S, K, I reduction machine built at Cambridge [4], ran combinator code directly in microcode, and later projects such as NORMA explored the idea further. Graph reduction turned out to run well enough on ordinary processors, and special hardware did not catch on.

**Supercombinators.**
Turner's combinators are fixed, so a program needs many small steps. Hughes [6] proposed instead to turn each function of the program into its own combinator, a *supercombinator*, by lifting out its free variables. One reduction then does the work of a whole function body.

**The G-machine.**
Johnsson and Augustsson compiled each supercombinator into code for an abstract machine that builds and rewrites the graph, the G-machine [7, 1], and from there to native code. That was the basis of the Lazy ML compiler and of the textbook by Peyton Jones [10], which remains the best introduction to the whole area, together with its practical companion [12].

**The STG machine.**
GHC, the main Haskell compiler, still reduces graphs, but its Spineless Tagless G-machine [11] avoids building the spine as a separate structure and represents every node as code that knows how to evaluate itself.

| | code (cells) | | reductions | |
|---|---|---|---|---|
| program | S K I | Turner | S K I | Turner |
| `fac.lam` | 29 | 14 | 192 | 87 |
| `fib.lam` | 37 | 19 | 503,481 | 240,795 |
| `church.lam` | 202 | 52 | 47,445 | 16,918 |
| `primes.lam` | 256 | 65 | 2,016,222 | 154,682 |

**Table 5:** Code size and reductions with S, K, I only (plus the K simplification), and with Turner's rules, from `prototype.py` with and without `--ski`. All four programs are written as definitions; the cells do not include one per definition.

**Combinators again.**
Bracket abstraction itself has never gone away. Kiselyov [8] gives algorithms whose code size grows linearly with the program, and Augustsson's MicroHs, a small Haskell implementation, compiles Haskell to combinators and runs them on a graph reducer, much as Turner did.

## 12 Summary

Combinator graph reduction removes the variables from a program before running it. Bracket abstraction translates each $\lambda x.M$ into a term built from a fixed set of combinators, using the rules $[x]\,x = \mathsf{I}$, $[x]\,e = \mathsf{K}\,e$ and $[x]\,(e_1\,e_2) = \mathsf{S}\,([x]\,e_1)\,([x]\,e_2)$, and Turner's extra combinators B, C, S′, B*, C′ keep the result small. The machine then needs no environments and no closures. It unwinds the spine to find the leftmost outermost redex, rewrites the root of the redex in place according to the combinator's rule, and repeats. Because arguments are shared and roots are overwritten, every expression is reduced at most once and only when needed, which is lazy evaluation. Y becomes a cycle, IF needs no thunks, and infinite lists come for free. The machine is small and regular, its only garbage-collection roots are the cells on the spine, and in our measurements it is the fastest of the three machines on higher-order programs and 2.4 times slower than the CAM on first-order arithmetic.

## Exercises

**Exercise 1.** Compile $\lambda f\,g\,x.\,f\,(g\,x)$ by hand, first with the three basic rules and then with Turner's. Check your answer by running `echo '\f g x.f (g x)' | python3 lam2graph.py -t`. Do the same for $\lambda f\,x\,y.\,f\,y\,x$.

**Exercise 2.** Extend the proof of theorem 1 to Turner's rules: show that each simplification of section 3.3 preserves the property $([x]\,e)\,x \;\to^*\; e$. Which of them relies on $\eta$-conversion, and why is that harmless for a program that is only run, never compared?

**Exercise 3.** Trace $(\lambda x\,y.\,y\,x)\;3\;(\lambda z.*\;z\;z)$ by hand. It compiles to C I 3 (S ∗ I) and takes 5 reductions. Check with `prototype.py --trace`. Where is the 3 shared?

**Exercise 4.** The rules overwrite the *root* of a redex. Suppose an implementation built a fresh cell for the result instead and left the root alone. Find a program for which it computes the same result but does more reductions, and explain why. What property of the program decides how much is lost?

**Exercise 5.** Add a combinator SEQ with the rule $\mathsf{SEQ}\,a\,b \;\to\; b$, after reducing $a$ to WHNF, and a source form `seq a b` for it. Use it to fix the space leak of section 8.3. How many reductions does the loop take for $n = 3000$?

**Exercise 6.** After every rewrite, the reducer pops the redex and unwinds again from its root, pushing the root again. For which rules is the new head of the root known without unwinding? Estimate from table 4 how many of the 70 instructions per reduction this would save in *fib*(20).

**Exercise 7.** The reducer checks for stack overflow on every push. Show that it would be enough to check once per reduction and at every nested call of `whnf`, provided the check leaves room for a bounded number of pushes. What is the bound, and why does it depend on the program?

**Exercise 8.** **rec** $x.\,x$ is detected because the indirection points to itself. **rec** $x.\,+\,x\ 1$ is just as meaningless, but the check does not catch it: the machine nests deeper and deeper until it runs out of stack. Explain why. GHC handles such cases with *black holes*: a cell is marked when its evaluation starts, and reaching a marked cell is an error. Design that for `graph.asm`. Which cells need marking?

**Exercise 9.** Write *map*, *take* and *sum* for the list encoding of section 8, and use them to compute the sum of the squares of the first 20 primes. Is the sum reduced before or after the primes are generated?

**Exercise 10.** Turner's languages did input and output with lazy lists: the program is a function from the input characters to the output characters. Design a version of `graph.asm` that prints the elements of a result list as they are produced, so that `primes.lam` could print primes forever. Which parts of the reducer change, and which roots must the collector know about?

# 13 Reference card

**Bracket abstraction** (compile the body of each $\lambda x.M$ first, then abstract $x$)

| | |
|---|---|
| $[x]\,x = \mathsf{I}$ | $[x]\,e = \mathsf{K}\,e$ (any other leaf) |
| $[x]\,(e_1\,e_2) = \mathsf{S}\,([x]\,e_1)\,([x]\,e_2)$, simplified: | |
| $\mathsf{S}\,(\mathsf{K}\,p)\,(\mathsf{K}\,q) = \mathsf{K}\,(p\,q)$ | $\mathsf{S}\,(\mathsf{B}\,p\,q)\,(\mathsf{K}\,r) = \mathsf{C}'\,p\,q\,r$ |
| $\mathsf{S}\,(\mathsf{K}\,p)\,\mathsf{I} = p$ | $\mathsf{S}\,p\,(\mathsf{K}\,q) = \mathsf{C}\,p\,q$ |
| $\mathsf{S}\,(\mathsf{K}\,p)\,(\mathsf{B}\,q\,r) = \mathsf{B}^*\,p\,q\,r$ | $\mathsf{S}\,(\mathsf{B}\,p\,q)\,r = \mathsf{S}'\,p\,q\,r$ |
| $\mathsf{S}\,(\mathsf{K}\,p)\,q = \mathsf{B}\,p\,q$ | |
| **if** $c\ t\ e = \mathsf{IF}\,c\,t\,e$ | **rec** $f.M = \mathsf{Y}\,(\lambda f.M)$ |

**Rules** (the root of the redex is overwritten; $\to \mathsf{I}\,v$ means an indirection)

| | |
|---|---|
| $\mathsf{I}\,x \to x$ | $\mathsf{Y}\,f \to f\,(\mathsf{Y}\,f)$, as a cycle |
| $\mathsf{K}\,x\,y \to x$ | $\mathsf{S}'\,k\,f\,g\,x \to k\,(f\,x)\,(g\,x)$ |
| $\mathsf{S}\,f\,g\,x \to f\,x\,(g\,x)$ | $\mathsf{B}^*\,k\,f\,g\,x \to k\,(f\,(g\,x))$ |
| $\mathsf{B}\,f\,g\,x \to f\,(g\,x)$ | $\mathsf{C}'\,k\,f\,g\,x \to k\,(f\,x)\,g$ |
| $\mathsf{C}\,f\,g\,x \to f\,x\,g$ | $\mathsf{IF}\,c\,t\,e \to t$ (if $c \neq 0$) or $e$, reducing $c$ first |

$+\,a\,b \to a + b$, reducing $a$ and $b$ first; likewise $-\ *\ /\ \%$, and $=, <$ giving 1 or 0

**Machine**

Unwind: from the root, push each cell and follow its function half, until a leaf.
A combinator with $n$ arguments and $\geq n$ cells on the spine: rewrite the $n$th cell, pop $n$, unwind from it.
Fewer cells: WHNF, a partial application. A number with an empty spine: the result.

# References


[1] L. Augustsson. A compiler for lazy ML. In *ACM Symposium on LISP and Functional Programming*, pages 218–227, 1984.

[2] G. Cousineau, P.-L. Curien, and M. Mauny. The categorical abstract machine. *Science of Computer Programming*, 8(2):173–202, 1987.

[3] C. J. Cheney. A nonrecursive list compacting algorithm. *Communications of the ACM*, 13(11):677–678, 1970.

[4] T. J. W. Clarke, P. J. S. Gladstone, C. D. MacLean, and A. C. Norman. SKIM — the S, K, I reduction machine. In *ACM Conference on LISP and Functional Programming*, pages 128–135, 1980.

[5] H. B. Curry and R. Feys. *Combinatory Logic*, volume I. North-Holland, 1958.

[6] R. J. M. Hughes. Super-combinators: a new implementation method for applicative languages. In *ACM Symposium on LISP and Functional Programming*, pages 1–10, 1982.

[7] T. Johnsson. Efficient compilation of lazy evaluation. In *SIGPLAN Symposium on Compiler Construction*, pages 58–69, 1984.

[8] O. Kiselyov. $\lambda$ to SKI, semantically. In *Functional and Logic Programming (FLOPS)*, volume 10818 of *LNCS*, pages 33–50. Springer, 2018.

[9] P. J. Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4):308–320, 1964.

[10] S. L. Peyton Jones. *The Implementation of Functional Programming Languages*. Prentice Hall, 1987.

[11] S. L. Peyton Jones. Implementing lazy functional languages on stock hardware: the Spineless Tagless G-machine. *Journal of Functional Programming*, 2(2):127–202, 1992.

[12] S. L. Peyton Jones and D. R. Lester. *Implementing Functional Languages: a Tutorial*. Prentice Hall, 1992.

[13] M. Schönfinkel. Über die Bausteine der mathematischen Logik. *Mathematische Annalen*, 92:305–316, 1924.

[14] D. A. Turner. A new implementation technique for applicative languages. *Software—Practice and Experience*, 9(1):31–49, 1979.

[15] D. A. Turner. Another algorithm for bracket abstraction. *Journal of Symbolic Logic*, 44(2):267–270, 1979.